Neural-operator wave simulations

FNO, DeepONet, and PFNO on the 1D elastic wave equation

SciML Project — wave equation operator baselines

July 31, 2026

Summary. Three neural operators are trained to map a heterogeneous material profile and an initial pulse to the complete space–time displacement field of the 1D elastic wave equation, supervised on finite-difference solutions. An earlier version of this comparison produced animations that showed noise rather than waves; the cause was diagnosed as a chronically under-specified training set (16 samples, 12 epochs), not a defect in the plotting. After enlarging the dataset to 512 samples, adding a one-cycle learning-rate schedule, and increasing model capacity, validation error falls from 81.9% to **2.07**% (FNO), 94.8% to **4.26**% (PFNO), and 97.3% to **14.43**% (DeepONet). This document explains the physics, the data, the architectures, the diagnosis, the visualization design, and the code. A second arm (Section 10) replaces the initial pulse with a Ricker point source and reports initial- and boundary-condition residuals, which turn out to need careful reading: the worst model scores the best boundary residual, because a collapsed field satisfies the condition trivially. Section 11 then promotes those residuals to training losses: FNO improves on every axis, PFNO is unmoved, and DeepONet gets worse — for an underfitting model the constraints pull toward the trivial solution.

Contents

1 What the simulations show

Each animation in `simulations/` covers one **held-out** velocity model. FNO, DeepONet, and PFNO each receive the material profile and the initial pulse, and predict the *entire* displacement field $u(x,t)$ in a single forward pass — no time stepping and no PDE solve at inference. The animation then sweeps a cursor through the predicted time slices and compares them against the finite-difference (FD) solution of the same problem.

The purpose is to test *generalization*. These velocity models were never seen in training. A model that has genuinely learned the solution operator must reproduce the correct wave speed, the correct reflection and transmission amplitudes at each material interface, and the correct arrival times, for a material it has never encountered.

Four animations are provided, one per material family. Each is the **median-error** sample of its family, not the best case.

File	Material	FNO	DeepONet	PFNO
`...sample_75_homogeneous.gif`	homogeneous	0.13%	4.29%	0.55%
`...sample_275_two_layer.gif`	two-layer	1.27%	8.20%	2.90%
`...sample_353_smooth.gif`	smooth	0.63%	10.70%	2.43%
`...sample_398_layered.gif`	layered (3–7)	4.67%	24.91%	10.63%

Percentages are relative L_2 error against the FD reference for that sample.

A further nine animations cover the three real velocity models, in `simulations/real/` — see §12.5. Four more in `simulations/source_term/` belong to the forced arm, and `simulations/superseded_undertrained/` keeps three from the original under-trained run.

2 How to read a frame

The figure is two rows by four columns.

velocity model $c(x)$	FNO vs FD ref	DeepONet vs FD ref	PFNO vs FD ref
FD reference field $u(x,t)$	FNO prediction	DeepONet prediction	PFNO prediction

- **Top left** — the wave speed $c(x) = \sqrt{E(x)/\rho(x)}$ for this sample, with the current time beside it. This is the *input* the operators are conditioned on; everything else in the figure follows from it.
- **Top row, columns 2–4** — displacement $u(\cdot,t)$ at the current time. The thick grey curve is the FD reference; the coloured curve is the model. Where the colour hides the grey, the model is correct. The title gives the relative L_2 for the whole field, not just that frame.
- **Bottom left** — the FD reference field over the full (x,t) domain, with a horizontal cursor at the current time.
- **Bottom row, columns 2–4** — each model's predicted field on the *same* colour scale, so panels are directly comparable.

Reading the space–time panels. Time runs upward. A wave at constant speed traces a straight line of slope $1/c$. The initial pulse at $t = 0$, $x = 0$ splits into a left-going and a right-going wave — the characteristic "V". Where $c(x)$ changes the V *kinks* (refraction changes the slope) and sheds a fainter secondary line travelling the other way (reflection). The layered sample shows this most clearly; the homogeneous sample is a clean straight V with no reflections at all.

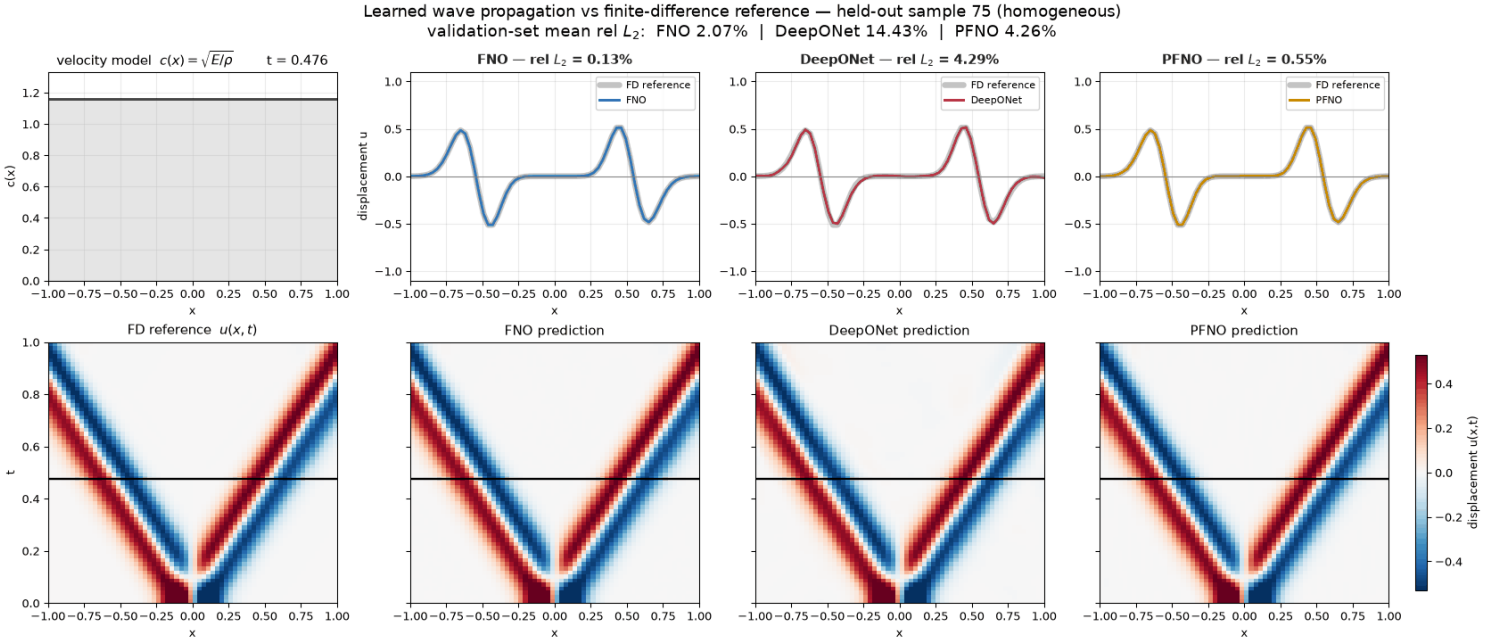

Figure 1: Homogeneous medium, c constant. A clean straight V with no reflections; FNO and PFNO are visually indistinguishable from the reference (0.13% and 0.55%).

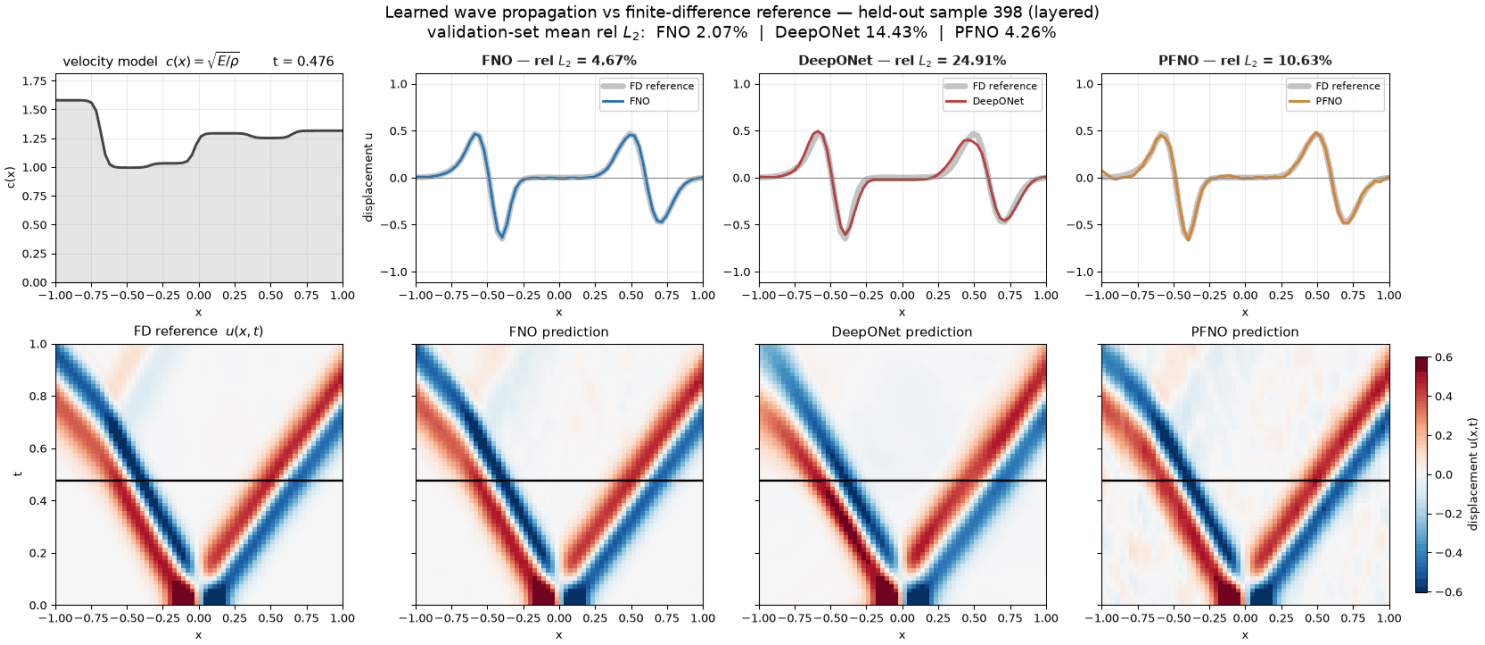

Figure 2: Layered medium, the hardest family. Every interface splits the wave, producing the secondary lines visible in the space–time panels. DeepONet's amplitude deficit against the grey reference is clearly visible in the top row.

3 The physics being learned

The governing equation is the 1D elastic wave equation in **conservative form**,

$$\rho(x)\, u_{tt} \;=\; \frac{\partial}{\partial x}[E(x)\, u_x]\,, \qquad x \in [-1,1], \quad t \in [0,1], \tag{1}$$

with $E(x)$ the stiffness and $\rho(x)$ the density. The local wave speed is $c(x) = \sqrt{E(x)/\rho(x)}$.

The conservative form matters. Expanding gives

$$\rho\, u_{tt} \;=\; E\, u_{xx} + E_x\, u_x, \tag{2}$$

so the naive $u_{tt} = c(x)^2 u_{xx}$ drops the $E_x u_x$ term — precisely the term that sets reflection and transmission amplitudes at a material interface. In layered media that approximation gets the physics wrong, so both the FD reference and the wider project use the conservative form throughout.

Initial conditions. A Gaussian-derivative pulse normalized to unit peak, with zero initial velocity,

$$f(x) = \frac{\partial}{\partial x} \exp\!\left(-\tfrac{1}{2}\left(\tfrac{x-x_0}{\sigma_g}\right)^2\right) \Big/ \max|\cdot|, \qquad g(x) = u_t(x,0) = 0, \tag{3}$$

with $\sigma_g = 0.1$ and $x_0 = 0$ fixed for every sample (hence `fixedic` in the dataset filename). Because the initial velocity is zero, the pulse splits symmetrically into two counter-propagating waves of roughly half amplitude each — which is why the $t = 0$ frame is about $1.8\times$ taller than everything after it. This has direct consequences for the colour scale (Section 13).

Boundary conditions. First-order absorbing (radiation) conditions, $u_t \mp c\, u_x = 0$ at the left/right boundary, so waves leave the domain without reflecting.

The FD reference. `wave/fd_solver.py` is an explicit second-order leapfrog scheme in flux form. Two details are load-bearing:

- **Harmonic-mean interface stiffness**, $E_{i+1/2} = 2E_iE_{i+1}/(E_i + E_{i+1})$, which preserves correct transmission and reflection across a jump in E.
- **Mur discretization** of the absorbing boundaries, $u_0^{n+1} = u_1^n + \beta\,(u_1^{n+1} - u_0^n)$ with $\beta = (c\,\Delta t - \Delta x)/(c\,\Delta t + \Delta x)$. The apparently natural centred-time / one-sided-space discretization is *unconditionally unstable* — round-off grows exponentially from the boundary.

The solver chooses its own Δt from the CFL condition; the generator then interpolates onto the uniform 64-point output time grid.

4 The dataset

`operator_data/wave_operator_fixedic_n512_nx64_nt64_t1_seed42.npz`

512 samples on a 64×64 (x,t) grid, $x \in [-1,1]$, $t \in [0,1]$, seed 42, CFL 0.35. Generation takes about two seconds.

Velocity models. $\rho(x) = 1$ everywhere, so $c(x) = \sqrt{E(x)}$. Each sample draws one of four $E(x)$ families uniformly:

Family	Count	Construction
`homogeneous`	125	constant, $E \sim U(0.7, 2.2)$
`two_layer`	130	tanh-smoothed step, left $U(0.7, 1.5)$ to right $U(1.0, 2.5)$, interface at $U(-0.45, 0.45)$, width $U(0.015, 0.06)$
`layered`	131	3–7 tanh-blended layers, values $U(0.7, 2.5)$, boundaries jittered by $\mathcal{N}(0, 0.04)$
`smooth`	126	base $U(0.9, 1.4)$ plus three sinusoids (freq. 1,2,3; amp. $U(-0.18, 0.18)$), clipped to $[0.55, 2.5]$

Across the 512 samples the realized wave speeds span $c \in [0.78, 1.58]$.

```
def sample_material_profile(rng, x):
    kind = rng.choice(["homogeneous", "two_layer", "layered", "smooth"])
    rho = np.ones_like(x, dtype=np.float64)

    if kind == "homogeneous":
        E = np.full_like(x, rng.uniform(0.7, 2.2), dtype=np.float64)
    elif kind == "two_layer":
        left, right = rng.uniform(0.7, 1.5), rng.uniform(1.0, 2.5)
        boundary, width = rng.uniform(-0.45, 0.45), rng.uniform(0.015, 0.06)
        alpha = 0.5 * (1.0 + np.tanh((x - boundary) / width))
        E = left * (1.0 - alpha) + right * alpha
    elif kind == "layered":
        n_layers = int(rng.integers(3, 8))
        values = rng.uniform(0.7, 2.5, size=n_layers)
        boundaries = np.linspace(x.min(), x.max(), n_layers + 1)[1:-1]
        boundaries += rng.normal(scale=0.04, size=boundaries.shape)
        width = rng.uniform(0.015, 0.05)
        E = np.full_like(x, values[0], dtype=np.float64)
        for value, boundary in zip(values[1:], boundaries):
            alpha = 0.5 * (1.0 + np.tanh((x - boundary) / width))
            E = E * (1.0 - alpha) + value * alpha
    else:  # smooth
        E = np.full_like(x, rng.uniform(0.9, 1.4), dtype=np.float64)
        for freq in (1, 2, 3):
            E += rng.uniform(-0.18, 0.18) * np.sin(
                freq * np.pi * (x + 1.0) + rng.uniform(0.0, 2.0 * np.pi))
        E = np.clip(E, 0.55, 2.5)

    return E.astype(np.float64), rho, str(kind)
```

Tensor layout. Inputs are (sample, channel, x, t) with five channels $[E, \rho, g, x, t]$; the three profiles are broadcast along the time axis and x, t are coordinate meshes. Outputs are (sample, 1, x, t).

```
inputs :  (512, 5, 64, 64)   channels [E(x), rho(x), g(x), x, t]
outputs:  (512, 1, 64, 64)   u(x,t)
```

5 The operator-learning problem

The models learn the map

$$[E(x),\ \rho(x),\ g(x),\ x,\ t] \longmapsto u(x,t), \tag{4}$$

supervised on FD solutions. This is *supervised* operator learning: there is no PDE residual in the loss, unlike the PINN/KAN side of this repository. The network never sees the equation, only input/output pairs.

Split. 20% validation via a seeded permutation: **410 train, 102 validation**. All reported numbers are on the validation set, and all four animated samples are validation samples.

Normalization. Per-channel mean/standard deviation for inputs and a single scalar mean/standard deviation for the output, both computed on the **training split only** so no validation statistics leak.

```
x_mean = X[train_idx].mean(dim=(0, 2, 3), keepdim=True)
x_std  = X[train_idx].std(dim=(0, 2, 3), keepdim=True).clamp_min(1e-6)
y_mean = Y[train_idx].mean()
y_std  = Y[train_idx].std().clamp_min(1e-6)
```

Loss and metric. Training minimizes MSE in normalized space. The reported metric is relative L_2 *in physical units*, computed per sample and averaged:

$$\text{rel } L_2 \;=\; \frac{\|\hat{u} - u\|_2}{\|u\|_2}. \tag{5}$$

This metric has a useful calibration property: **a model that outputs all zeros scores about** 100%. Anything near 100% has learned nothing. The state with the best validation MSE across all epochs is kept, not the final state.

6 The three architectures

6.1 FNO — 2D Fourier Neural Operator

Lift the five input channels to `width` channels with a 1×1 convolution, apply blocks of GELU(spectral(z) + local(z)), then project to one output channel. The spectral layer takes a 2D real FFT over (x, t), multiplies the lowest modes$_x \times$ modes$_t$ coefficients by learned complex weights, zeroes the rest, and inverts.

```
def forward(self, x):
    batch, _, nx_local, nt_local = x.shape
    x_ft = torch.fft.rfft2(x, norm="ortho")
    out_ft = torch.zeros(batch, self.weight_pos.shape[1], nx_local,
                         nt_local // 2 + 1, dtype=torch.cfloat, device=x.device)
    mx = min(self.modes_x, nx_local)
    mt = min(self.modes_t, nt_local // 2 + 1)
    # Both ends of the x axis are kept: rfft2 is only half-spectrum in t.
    out_ft[:, :, :mx, :mt]  = self.complex_mul(x_ft[:, :, :mx, :mt],
                                               self.weight_pos[:, :, :mx, :mt])
    out_ft[:, :, -mx:, :mt] = self.complex_mul(x_ft[:, :, -mx:, :mt],
                                               self.weight_neg[:, :, :mx, :mt])
    return torch.fft.irfft2(out_ft, s=(nx_local, nt_local), norm="ortho")
```

Truncating to low modes is the regularizer that makes the operator resolution-independent: the learned kernel is a function, not a grid. Because the wave equation is close to diagonal in Fourier space, and because this architecture sees the *joint* (x, t) spectrum, FNO is the natural fit — and it wins by a wide margin.

6.2 DeepONet

A branch network encodes the input functions, a trunk network encodes the query coordinates, and the output is their inner product:

$$u(x,t) \;\approx\; \frac{1}{\sqrt{p}} \sum_{k=1}^{p} b_k([E, \rho, g])\; \tau_k(x,t) \;+\; \text{bias}. \tag{6}$$

```
def forward(self, x):
    branch_input = x[:, :3, :, 0].reshape(x.shape[0], -1)   # E, rho, g
    branch_features = self.branch(branch_input)             # (batch, latent)
    trunk_features = self.trunk(self.coordinates)           # (nx*nt, latent)
    values = torch.einsum("bp,np->bn", branch_features, trunk_features)
    values = values / math.sqrt(branch_features.shape[-1]) + self.bias
    return values.reshape(x.shape[0], 1, self.nx, self.nt)
```

This is a **rank-p separable expansion**: the entire space–time dependence must be spanned by p fixed basis functions $\tau_k(x,t)$, with the material only choosing coefficients. A sharp front whose position depends on $c(x)$ is expensive to represent this way, which is the structural reason DeepONet trails the two spectral models — not a bug.

6.3 PFNO — paralleled FNO

Naming. Here PFNO means *paralleled* Fourier neural operator, not *physics-informed*. The reference (Li et al., arXiv:2209.12340, included as `fno paper/2209.12340v3.pdf`) solves Helmholtz problems with one FNO per frequency. This is a time-domain analogue of that idea, not a reproduction of their 2D OpenFWI experiment.

Take the real FFT of the solution along time. Each temporal frequency bin gets its *own* small 1D FNO acting on the spatial profiles $[E, \rho, g, x]$, which predicts the real and imaginary parts of that bin. An inverse real FFT reassembles the field.

```
def forward(self, x):
    profiles = x[:, :4, :, 0]              # E, rho, g, x -- time-independent
    coefficients = []
    for frequency_id, branch in enumerate(self.frequency_branches):
        real_imag = branch(profiles)
        imag = real_imag[:, 1]
        # DC and Nyquist coefficients must be real for a real-valued signal.
        if frequency_id == 0 or (self.nt % 2 == 0
                                 and frequency_id == self.n_frequencies - 1):
            imag = torch.zeros_like(imag)
        coefficients.append(torch.complex(real_imag[:, 0], imag))
    spectrum = torch.stack(coefficients, dim=-1).unsqueeze(1)
    return torch.fft.irfft(spectrum, n=self.nt, dim=-1)
```

At $n_t = 64$ this instantiates $n_t/2 + 1 = 33$ independent branches, evaluated in a Python loop. That is why PFNO is by far the slowest to train (819 s versus 73 s for FNO) despite having six times fewer parameters.

7 Why the first version failed

The original animations showed noise rather than waves, and DeepONet's panel was essentially blank. The cause was not the plotting code — **the models were untrained**, and the plots were faithfully showing that.

The previous run's own metrics:

Model	Validation relative L_2
FNO	81.9%
PFNO	94.8%
DeepONet	97.3%

Since zero output scores about 100%, DeepONet at 97.3% had learned almost nothing; its "smooth, plausible" field was a near-zero field. Training times gave it away too: 0.18 s for DeepONet and 1.3 s for FNO — on an H100.

The configuration was **16 samples, 12 epochs, batch 4** — about 39 gradient steps in total — with 10–30 k parameter models. The repository's own prior runs already bracketed the data threshold:

Run	Samples	Grid	Parameters	Val relative L_2
`operator_results_pilot`	16	48^2	99 k	58.3%
`fno_t4_smoke`	32	64^2	99 k	94.8%
`fno_t4_medium`	256	128^2	4.7 M	7.4%
`fno_t4_full`	768	128^2	13.1 M	3.2%

Operator learning on this problem needs a few hundred samples. Sixteen is not a small dataset for this task; it is below the threshold at which anything is learnable at all.

8 What was changed

Three changes, in order of importance.

1. Dataset size: 16 → 512 samples (64×64 grid). The dominant factor. FD generation costs about two seconds, so the original 16-sample dataset was never a compute constraint — merely an under-specified one.

2. One-cycle learning-rate schedule. With a flat learning rate these models stall near the zero solution regardless of epoch budget: MSE in normalized space sits at ≈ 1.0, exactly what predicting the mean achieves. Warmup-then-anneal is what lets them escape.

```
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=LEARNING_RATE,
    total_steps=EPOCHS * len(train_loader), pct_start=0.15,
)
```

3. Capacity and budget.

	Before	**After**
Epochs	12	400
Batch size	4	16
Learning rate	2e-3 (flat)	3e-3 (one-cycle)
FNO	width 8, modes 6, layers 2	width 48, modes 20, layers 4
DeepONet	latent 32, hidden 64	latent 256, hidden 512
PFNO	width 8, modes 8, layers 2	width 24, modes 20, layers 3

Everything else — the physics, the FD solver, the tensor convention, the normalization scheme, the split procedure, the metric, and the model code itself — is unchanged.

9 Results

512 samples, 64×64, 400 epochs, AdamW (lr 3×10^{-3}, weight decay 10^{-4}), batch 16, one NVIDIA H100 NVL.

Model	**Parameters**	**Train time**	**Val MSE**	**Val relative L_2**	**Was**
FNO	7,387,297	73 s	5.87×10^{-5}	**2.07%**	81.9%
PFNO	1,225,290	819 s	1.79×10^{-4}	**4.26%**	94.8%
DeepONet	888,321	25 s	1.48×10^{-3}	**14.43%**	97.3%

For context, FNO at 2.07% on 512 samples beats this repository's own `fno_t4_full` baseline (3.21% on 768 samples at 128^2).

> **Read these against §12.2.** The targets in this table were solved on the same 64-point grid as the output, which is 13% away from a grid-converged reference. Retraining against a converged target leaves FNO and PFNO essentially unchanged (1.96% and 4.03%) but moves DeepONet from 14.43% to 24.57%. The validation split here is also random rather than spatial, which is harmless for synthetic samples drawn i.i.d. but not for real geology.

Family	**n**	**FNO**	**DeepONet**	**PFNO**
homogeneous	21	0.13%	6.07%	0.74%
smooth	30	0.93%	10.94%	2.59%
two-layer	26	1.86%	16.22%	3.96%
layered (3–7)	25	5.28%	23.78%	9.55%
all	**102**	**2.07%**	**14.43%**	**4.26%**

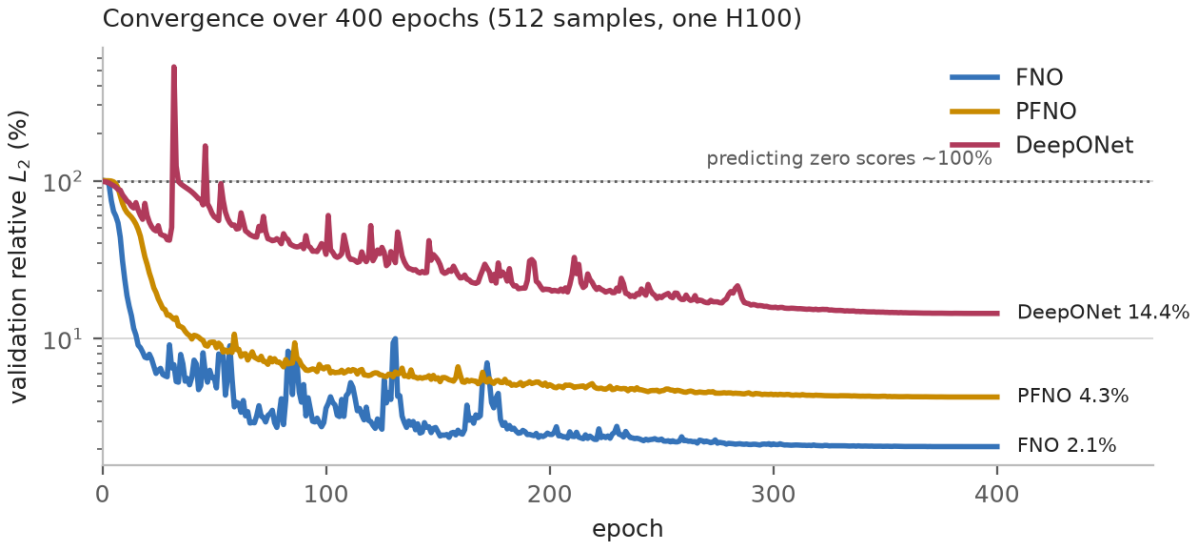

Figure 3: Convergence of the three operators. All three begin at ≈ 100% — the score of predicting zero — and the dotted line marks that level. FNO's visible oscillation is the one-cycle schedule at high learning rate; the best checkpoint by validation MSE is retained, not the final state.

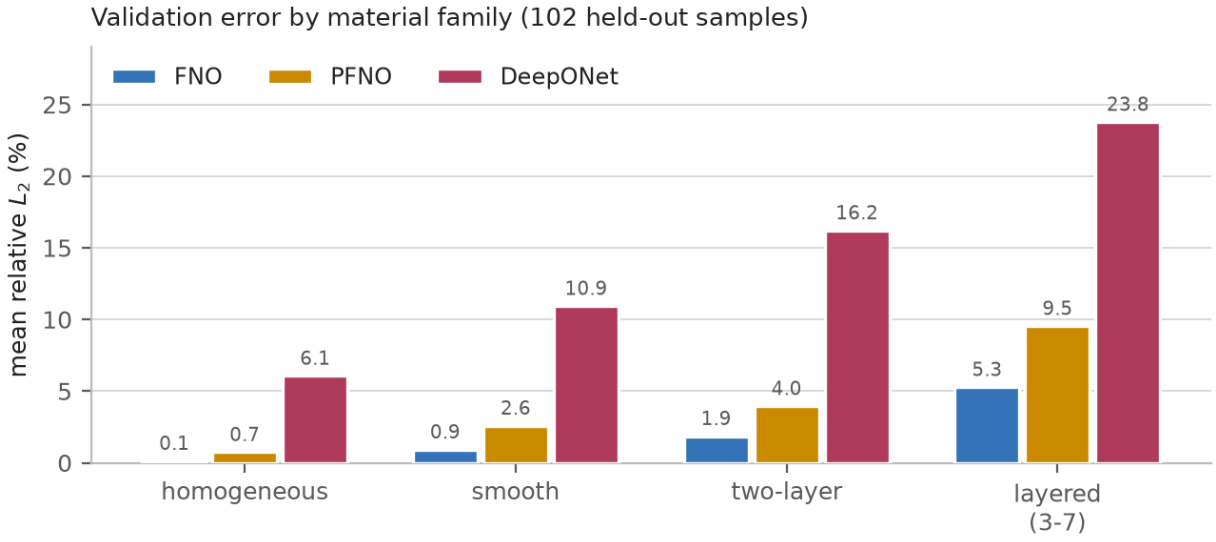

Figure 4: Validation error by material family. The ordering is identical for all three models and matches physical intuition: error grows with the number of interfaces, since each interface adds a reflected and a transmitted wave the operator must place correctly.

Homogeneous media — a straight V in space–time with no reflections — are reproduced nearly exactly. The model ranking (FNO < PFNO < DeepONet) holds in every family and is what the architectures predict: FNO sees the joint (x, t) spectrum; PFNO factorizes by temporal frequency and solves each bin with an independent 1D operator, losing the joint structure; DeepONet compresses the whole material dependence through a rank-p separable expansion.

10 The forced arm: a source term, and its IC/BC residuals

Everything above is driven purely by an initial pulse, with no forcing. This section adds a second, independent arm in which the wave is driven by a source term instead, and reports how well each model's output respects the initial and boundary conditions. The unforced arm is unchanged; the two run side by side.

10.1 Equation and source design

$$\rho(x)\, u_{tt} \;=\; \frac{\partial}{\partial x}[E(x)u_x] \;+\; \underbrace{s(x)\, w(t)}_{\text{source}}, \qquad x \in [-1, 1],\; t \in [0, 2] \tag{7}$$

The source is separable, so two input channels carry it exactly:

- $s(x) = \exp\!\big(-((x - x_s)/w_s)^2\big)$, a narrow Gaussian standing in for a point source, $w_s = 0.04$, position $x_s \sim U(-0.5, 0.5)$.
- $w(t)$ a Ricker wavelet, peak frequency $f \sim U(3, 6)$, shifted by $t_0 = 1.2/f$ so that $w(0) \approx -1.8 \times 10^{-5}$ and the start is quiescent to numerical precision.

The initial conditions become **quiescent**, $u(x, 0) = 0$ and $u_t(x, 0) = 0$ — verified exactly zero in the generated data. Boundaries remain first-order absorbing (Mur), unchanged from the unforced arm.

Why the window is $t \in [0, 2]$. At $t_{\max} = 1$ the waves only just reach the boundary: 99% of the field energy is still inside the domain at the final step, so the absorbing condition is barely exercised and a boundary residual would measure almost nothing. Extending to $t_{\max} = 2$ lets the waves fully exit — 99.8% of the energy has left by the final step — which is what makes a BC residual meaningful. The grid is 64×80; at $f \le 6$ that is 6.6–13 samples per wavelet period.

10.2 This is a substantially harder problem

In the unforced arm only $E(x)$ varied between samples: $\rho \equiv 1$, the initial pulse was fixed, and x, t are fixed meshes. The forced arm has **three** varying inputs:

Channel	Unforced arm	Forced arm
$E(x)$	varies	varies
$\rho(x)$	constant	constant
source / IC profile	constant	**varies** (x_s)
temporal signature	—	**varies** (f)
x, t	fixed meshes	fixed meshes

The results below should be read against that: the errors are much larger than the unforced arm's, and the two arms are *not* directly comparable (different grid, time window, and physics).

Model	Parameters	Train time	Forced rel L_2	(Unforced)
FNO	7,387,345	109 s	**17.13**%	2.07%
PFNO	1,524,298	1273 s	**31.89**%	4.26%
DeepONet	929,281	40 s	**91.82**%	14.43%

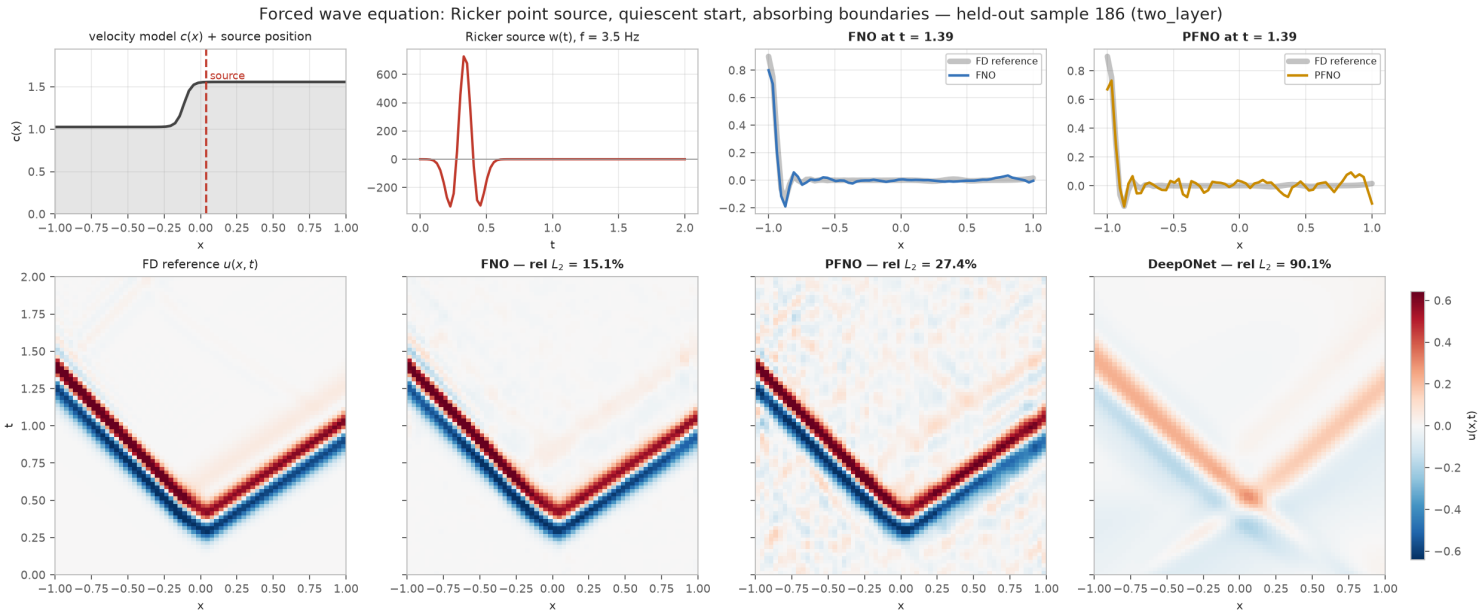

Figure 5: Forced arm, a two-layer sample. The source position (dashed red) and the Ricker wavelet are now inputs. FNO tracks the reference; PFNO reproduces the arrival but fills the quiet region with speckle; DeepONet produces a smeared blob where the reference has sharp arrivals.

10.3 Definitions of the residuals

These are **diagnostics, not training objectives** — the loss is still plain supervised MSE, with no physics term. They answer a question the field error cannot: does the output respect the conditions it was supposed to inherit?

Initial-condition residuals. With a quiescent start the targets are zero, so the residuals are the predicted values themselves:

$$R^{u}_{\mathrm{IC}} = \sqrt{\tfrac{1}{n_x}\textstyle\sum_i \hat{u}(x_i,0)^2}, \qquad R^{u_t}_{\mathrm{IC}} = \sqrt{\tfrac{1}{n_x}\textstyle\sum_i \hat{u}_t(x_i,0)^2}. \tag{8}$$

Boundary-condition residuals. The absorbing conditions should annihilate an outgoing wave:

$$r_L(t) = \hat{u}_t(-1,t) - c(-1)\,\hat{u}_x(-1,t), \qquad r_R(t) = \hat{u}_t(+1,t) + c(+1)\,\hat{u}_x(+1,t), \tag{9}$$

reported as an absolute RMS over t and as a scale-free relative version

$$\mathrm{BC}_{\mathrm{rel}} = \frac{\mathrm{RMS}_t(r)}{\mathrm{RMS}_t(\hat{u}_t) + \mathrm{RMS}_t(c\,\hat{u}_x)}. \tag{10}$$

All derivatives use second-order finite differences on the output grid (one-sided at edges).

The reference floor is essential. The finite-difference solution satisfies these conditions only to discretisation accuracy on the coarse output grid — first-order Mur, plus interpolation from the solver's internal time step. Its own residual is therefore the achievable floor, and every model number below is meaningless without it. The metric was validated on two controls: a standing wave, which violates the outgoing condition maximally, scores $\mathrm{BC}_{\mathrm{rel}} = 1.0000$ exactly; and an all-zero field, which satisfies both conditions trivially, scores 0.

10.4 Results

	IC displacement	IC velocity	BC absolute	BC relative
FD reference	0	3.9×10^{-4}	0.886	0.126
FNO	4.0×10^{-3}	0.183	1.500	0.249
PFNO	3.2×10^{-2}	1.185	3.691	0.515
DeepONet	3.3×10^{-2}	0.269	0.123	**0.144**

10.5 A low boundary residual is not evidence of good physics

DeepONet reports the *best* boundary score — 0.144 relative, and an absolute residual of 0.123, seven times *smaller* than the reference's own 0.886. It is also by far the worst model, at 91.8% field error.

This is not a contradiction; it is the failure mode the all-zero control predicted. Decomposing the two terms that are supposed to cancel at the left boundary, averaged over the validation set:

	RMS u_t	RMS $c\,u_x$	RMS residual
FD reference	3.285	4.051	0.902
FNO	2.817	3.790	1.524
PFNO	2.793	4.509	3.603
DeepONet	**0.561**	**0.583**	0.127

DeepONet's boundary terms are about six times smaller than the reference's, and its whole-field RMS is 0.41× the reference's. There is barely a wave arriving at its boundary at all, so it satisfies the radiation condition by having nothing to radiate. **IC and BC residuals are therefore not a standalone quality measure** — they are only interpretable alongside the field error.

Among the models that do produce a wave of the right amplitude, the residuals rank as the field error does: FNO at roughly twice the reference floor, PFNO at about four times.

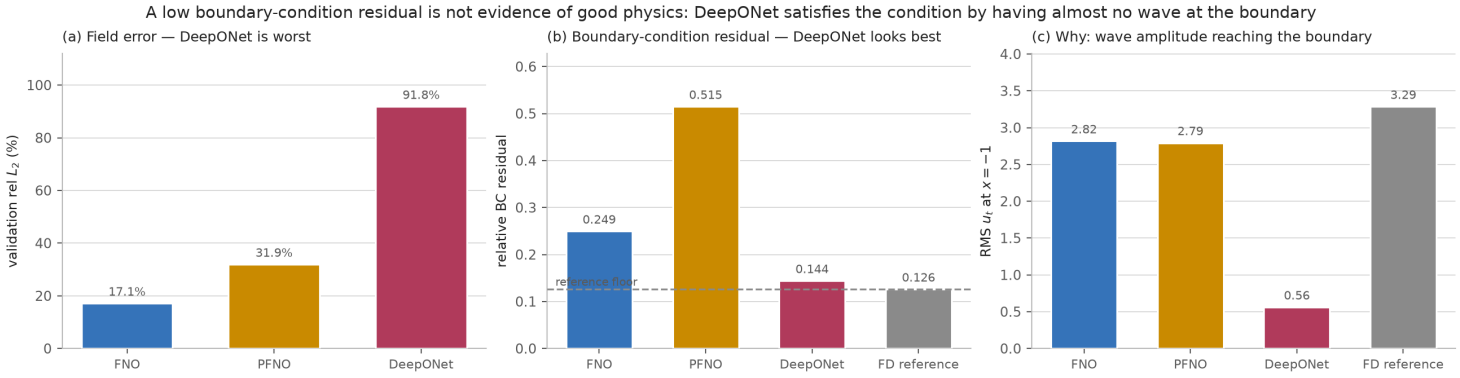

Figure 6: The same three models, three ways. DeepONet is worst on field error (a) yet best on boundary residual (b); panel (c) is the explanation — the wave amplitude reaching its boundary is a sixth of the reference's.

10.6 PFNO's initial-velocity residual

One result is disproportionate rather than merely ordered. PFNO's IC velocity residual is 1.185, six times FNO's 0.183, although its field error is only 1.9× FNO's.

The cause is structural. PFNO assembles the field from $n_t/2 + 1 = 41$ independent temporal-frequency branches. Enforcing $u_t(x, 0) = 0$ requires the phases of all 41 bins to agree precisely at $t = 0$; small independent per-bin errors barely perturb the field but accumulate directly in its time derivative. The speckle filling the quiet region of PFNO's panel in the figure above is the same effect seen in the field.

11 Making IC and BC actual training losses

Section 10 treats the residuals purely as diagnostics: the loss is supervised MSE and nothing about the physics reaches the gradients. This section adds them to the objective,

$$\mathcal{L} = \underbrace{\mathrm{MSE}(\hat{y}, y)}_{\text{data}} + \lambda\Big(\mathcal{L}^{u}_{\mathrm{IC}} + \mathcal{L}^{u_t}_{\mathrm{IC}} + \mathcal{L}_{\mathrm{BC}}\Big), \tag{11}$$

which is the PINN-style treatment already used on the coordinate-network side of this repository (`src/losses/wave_loss.py`).

Implementation. `physics_metrics.py` is numpy and cannot backpropagate, so the same quantities are reimplemented in torch in `physics_losses_torch.py`. The two agree to $\sim 10^{-8}$, float32 precision — verified before any of the results below were trusted. Each term is divided by a fixed scale precomputed from the training targets (RMS of u, of u_t, and of the boundary terms), so all three land in a comparable range and a single λ balances them against the data term. The scales are constants, not batch statistics: a data-dependent denominator inside a loss makes gradients noisy. Setting `--lambda-physics 0` reproduces the supervised run exactly.

11.1 Weight sweep

λ	Field error	IC displacement	IC velocity	BC relative
0 (baseline)	17.13%	4.03×10^{-3}	0.1835	0.2494
0.003	17.02%	3.47×10^{-3}	0.1507	0.2380
0.03	16.92%	2.04×10^{-3}	0.0884	0.1928
0.3	15.89%	1.36×10^{-3}	0.0461	0.1205
1.0	14.84%	5.15×10^{-4}	0.0171	0.0657
3.0	14.20%	7.56×10^{-4}	0.0151	0.0426
10.0	**12.70%**	$\mathbf{1.95 \times 10^{-4}}$	**0.0058**	**0.0291**

FNO, 500 epochs per point. Every metric improves monotonically and **no trade-off appears** — not even at $\lambda = 10$, where the physics terms dominate the objective. Field error falls 26% in relative terms and the initial-velocity residual improves 32×.

That is unusual enough to be worth suspecting, so the collapse check from Section 10 was repeated: at $\lambda = 1$ the field RMS is 0.982× the reference, against 0.969× for the supervised baseline. The model is *more* faithful in amplitude, not less, so the gain is real rather than the degenerate solution reappearing. The likely reason is regularisation: with 410 training samples on a hard problem, the constraints supply true information about the solution that the data alone does not pin down.

11.2 Three architectures, three outcomes

All three trained at $\lambda = 1$ — a round value inside the beneficial regime, chosen over the sweep's best because PFNO's raw physics loss is about six times FNO's, so the same weight bites considerably harder there.

	Field error		IC velocity		BC relative	
	$\lambda = 0$	$\lambda = 1$	$\lambda = 0$	$\lambda = 1$	$\lambda = 0$	$\lambda = 1$
FNO	17.13%	**14.84%**	0.183	**0.017**	0.249	**0.066**
PFNO	31.89%	31.43%	1.185	0.833	0.515	0.386
DeepONet	91.82%	93.55%	0.269	0.148	0.144	0.144

FNO — improved on every axis. Its relative boundary residual, 0.066, is now half the finite-difference reference's own 0.126. That is not "better than the truth": the reference's residual comes from its first-order Mur discretisation, and the network is under no obligation to reproduce that particular error while still sitting 15% away in L_2.

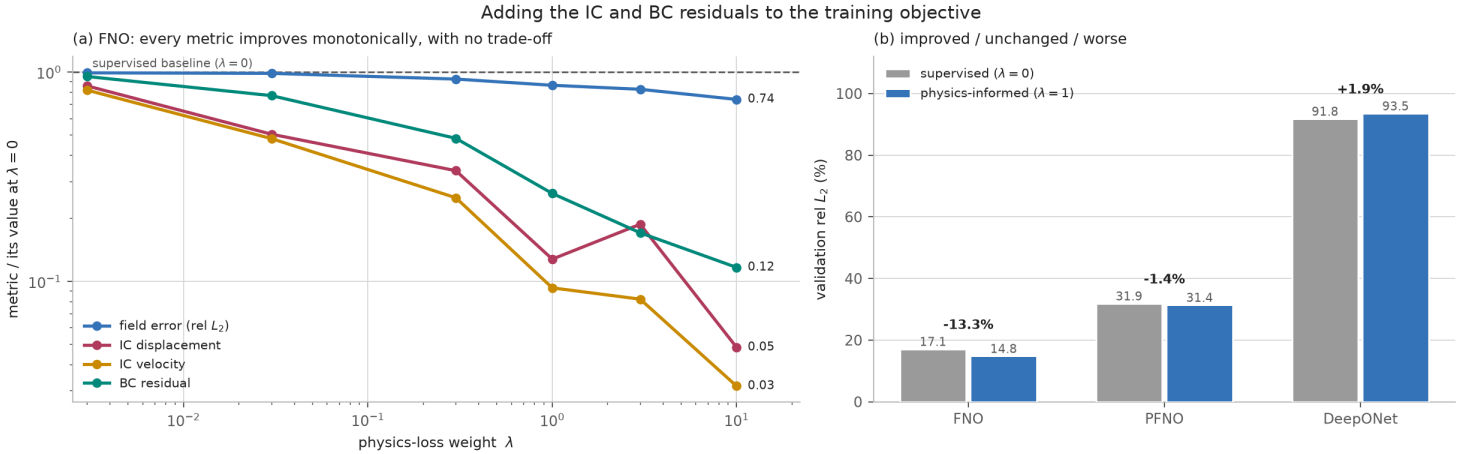

Figure 7: (a) Every metric, normalised by its own supervised baseline; below the dashed line is an improvement. (b) The same weight applied to all three architectures gives three different outcomes.

PFNO — essentially unchanged. This was the architecture predicted to benefit most, since its initial-velocity residual was its clearest structural weakness. It did not: field error moved $31.9 \to 31.4\%$ and the residual only $1.185 \to 0.833$. The prediction was wrong, and the reason is the more interesting result. PFNO's 41 frequency branches are independent networks with no mechanism to coordinate phase; a gradient on the assembled field cannot easily repair a coordination failure distributed across 41 separate sub-networks. The defect is architectural, not an optimisation shortfall.

DeepONet — worse. Field error rose $91.8 \to 93.6\%$ and the field amplitude fell further, $0.409 \to 0.374$ of the reference. This is the sharpest lesson of the section. Zero satisfies every one of these homogeneous constraints *exactly*, so for a model that already cannot fit the data the physics terms are a gradient pointing **toward** the trivial solution. Physics-informed training reinforces collapse when the model lacks the capacity to fit the data; the constraints only help once a model is good enough that satisfying them is non-trivial.

12 Real velocity models: Marmousi, Overthrust, Salt

Everything above draws $E(x)$ from a synthetic sampler — tanh steps and sums of sinusoids. This section replaces that sampler with 1D depth columns taken from three published velocity models, keeping the problem, the solver, the architectures and the training schedule identical.

What is real here, and what is not. The *material* is real, in the sense that these are the models the exploration-geophysics community uses as ground-truth benchmarks. They are not field measurements: Marmousi and Overthrust are both synthetic constructions designed to be geologically realistic. Well logs would be genuinely measured $V_p(z)$; these are a step short of that but far beyond tanh profiles. The *wave field* is still FD-simulated in every case, because no measured full-field $u(x,t)$ exists for this geometry — the target has to be computed either way.

Model	Geometry	Spacing	Density	V_p range	Source
`marmousi`	2301×739	4 m	measured	1532–5500 m/s	geoazur WIND
`overthrust`	$801 \times 801 \times 161$	25 m	none ($\rho = 1$)	2182–6000 m/s	SEG/EAGE
`salt`	$676 \times 676 \times 180$	20 m	none ($\rho = 1$)	1500–4482 m/s	SEG/EAGE

Water caps and constant basements are trimmed. Each sample takes a random contiguous depth window from a random trace, reduced onto the 64-point grid by **Backus averaging** — arithmetic mean of ρ, harmonic mean of the modulus $M = \rho V_p^2$, the correct long-wavelength effective medium for normal-incidence 1D propagation. Point-sampling a 739-sample column down to 64 would alias

the interfaces into noise: 3.2% of Marmousi's cells jump by more than 200 m/s, with a maximum of 1530 m/s in a single 4 m cell.

Wave speed is preserved exactly by the nondimensionalisation. Dividing by fixed per-model reference values (the medians V_p^{ref}, ρ^{ref}) gives $\tilde{c} = \sqrt{\tilde{E}/\tilde{\rho}} = V_p/V_p^{\mathrm{ref}}$, so genuine sample-to-sample speed variation survives instead of being normalised away.

12.1 Three methodological changes the real data forces

Density varies (Marmousi only). Marmousi ships a density model, so channel 1 is no longer all ones. Overthrust and Salt ship velocity only and get $\rho = 1$ like the synthetic arm. Gardner's relation could synthesise a density, but it is an empirical fit — it would add fiction while claiming realism.

The train/validation split must be spatial. Neighbouring Marmousi traces are 4 m apart and differ by a mean $|\Delta V_p|$ of 6 m/s — 0.2% of the mean. Even 20 traces apart the difference is only 3.7%. Drawing 512 samples at random from 2301 traces puts near-duplicates of training profiles into validation, and the resulting "generalisation" error is really an interpolation error. Validation instead takes four contiguous trace blocks spread across the model, with 320 m buffers discarded on each side; the realised minimum train-to-validation separation is 328 m (Marmousi), 400 m (Overthrust), 420 m (Salt). The split ships inside the dataset as a `split` array, and `train_operators.py` honours it when present, falling back to its random split otherwise.

The FD target has to be grid-converged. This turned out to matter far more than expected, and it applies to the synthetic arm too.

12.2 The target was not converged

The original datasets solve the FD reference on the same 64-point grid as the output. Measured against a `refine=32` reference:

refine	FD grid	relative L_2 vs converged	Cost / 512
1	64	13.35%	1.8 s
2	127	3.87%	2.9 s
4	253	1.14%	8.9 s
8	**505**	**0.32%**	**22 s**
16	1009	0.08%	41 s

Regenerating the synthetic dataset with **identical materials** (same seed, same sampler — the `inputs` arrays are bit-identical) and only a converged target moves the targets by **9.72**% on average, and by 7.91% even on `homogeneous` samples, which contain no interfaces at all. That part is not interface error: it is numerical dispersion of the initial pulse itself, whose width $\sigma_g = 0.1$ spans only about three cells at 64 points.

What happens when the models are retrained against the corrected target is the interesting part:

Model	vs `refine=1`	vs `refine=8`
FNO	2.07%	**1.96**%
PFNO	4.26%	**4.03**%
DeepONet	14.43%	**24.57**%

FNO and PFNO are essentially unchanged. The discretisation error is a deterministic function of the material, not noise, so a model with enough capacity simply learns whichever target it is given and scores the same against it. The published 2.07% was therefore never *inflated* — but it should be read as "2% away from the 64-point FD solution", not "2% away from the wave".

DeepONet is the exception: it nearly doubles. The `refine=1` target is smoother — dispersion smears the pulse — and a rank-p separable expansion $u \approx \sum_p b_p(E)\,\tau_p(x,t)$ represents smooth fields much more cheaply than sharp ones. The under-resolved target was flattering it. This is worth

stating plainly: one of the three architectures had its headline number materially improved by a numerical artefact, and the effect was invisible until the target was fixed.

12.3 Results

512 samples per arm, 400 epochs, AdamW (lr 3×10^{-3}, weight decay 10^{-4}), batch 16, NVIDIA H100 NVL. All four arms use `refine=8` targets; the three real arms use disjoint trace-block splits. Parameter counts are identical across arms (FNO 7,387,297; PFNO 1,225,290; DeepONet 888,321).

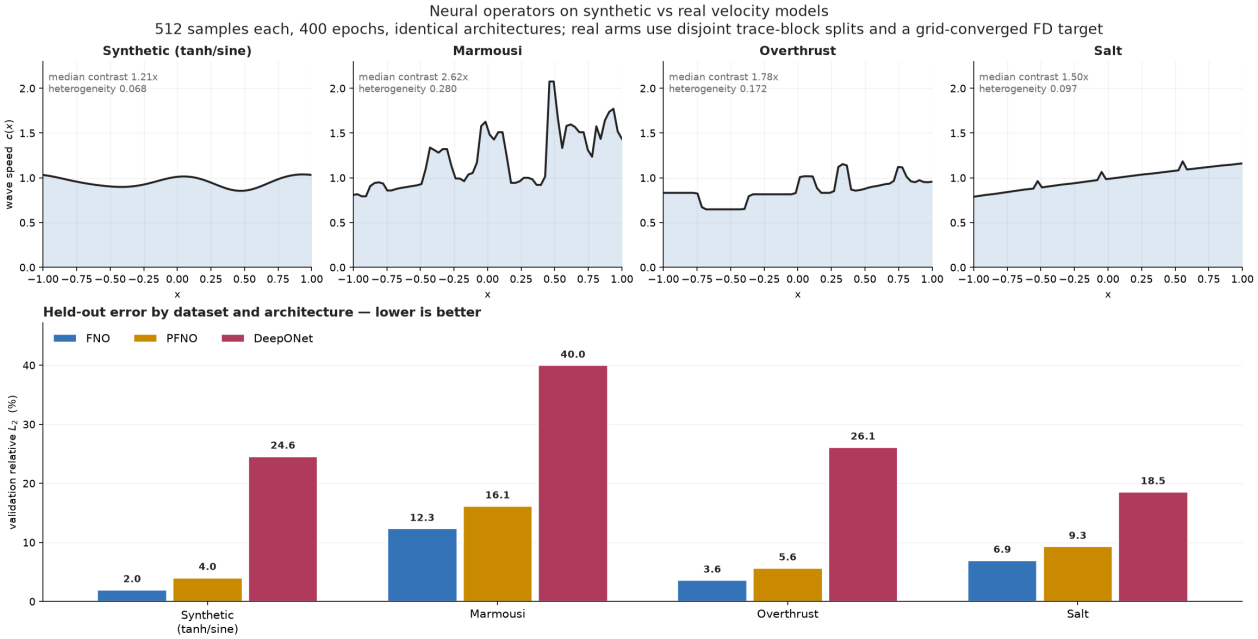

Figure 8: Held-out error by dataset and architecture. Top row: a median-contrast velocity profile from each arm, with its heterogeneity (median within-sample standard deviation of $c(x)$). The architectural ranking FNO < PFNO < DeepONet holds on every dataset, but the margins compress as the material stops being smooth.

Arm	Heterogeneity	Median contrast	FNO	PFNO	DeepONet
synthetic (tanh/sine)	0.068	1.21×	**1.96%**	**4.03%**	**24.57%**
Overthrust	0.172	1.78×	**3.59%**	**5.65%**	**26.11%**
Salt	0.097	1.50×	**6.95%**	**9.30%**	**18.55%**
Marmousi	0.280	2.62×	**12.33%**	**16.15%**	**39.97%**

The architectural ranking survives. FNO < PFNO < DeepONet on every arm, without exception. The conclusion drawn from synthetic data holds on real geology, which is the main thing this section was run to check.

The margins compress. On synthetic data FNO beats PFNO by 2.1×; on Marmousi by 1.3×. The gap that separates a joint (x, t) spectral operator from a per-frequency one narrows once the material stops being smooth — both are struggling with the same thing.

Difficulty tracks heterogeneity, not contrast. Marmousi is 4.1× more heterogeneous than the synthetic set and 6× harder for FNO. Overthrust, with a similar velocity range but much smoother columns, costs only 1.8×.

12.4 Contrast is a poor difficulty predictor

Grouping each arm's validation samples into per-model contrast terciles gives a result that does not generalise:

Arm	Low contrast	Moderate	High contrast
Marmousi	14.96%	12.16%	**9.87**%
Overthrust	5.02%	3.33%	**2.67**%
Salt	3.49%	2.88%	**13.26**%

(FNO; PFNO and DeepONet show the same shape.)

Marmousi and Overthrust get *easier* as contrast rises. That is not a normalisation artefact — across Marmousi's bands ‖target‖ rises only 2% while FNO's absolute L_2 error falls 33%. The plausible reading is that strong reflectors produce distinct, high-amplitude coherent arrivals that are structurally easy to learn, whereas low-contrast windows give weak diffuse scattering: subtle, low-amplitude deviations from a smooth background.

Salt reverses it completely — its high-contrast band is 4.6× worse than its moderate band. The explanation is geological rather than statistical: Salt's low- and moderate-contrast columns are mostly a smooth sediment velocity gradient, while the high-contrast columns are exactly the ones that intersect a salt body, where V_p jumps from ∼2000 to 4482 m/s across a sharp, isolated boundary. A single strong impedance jump is a different and harder problem than a gradual increase in range.

So $V_p^{\max}/V_p^{\min}$ summarises the wrong thing. What predicts difficulty is whether the column contains a sharp isolated reflector, not how wide its velocity range is. The per-model terciles behind the `kinds` labels are therefore useful only *within* a model — Marmousi's high band starts at 2.96× and Salt's at 1.61×; the datasets carry the absolute `contrast` array for cross-model work.

12.5 Animations

Nine GIFs in `simulations/real/<model>/`, one per contrast band per model, each the **median-error** sample of its band. Layout and colour scale are identical to the synthetic ones (§13), so they can be read side by side.

Model	Band	Sample	FNO	PFNO	DeepONet
Marmousi	low	412	12.74%	13.49%	40.55%
Marmousi	moderate	458	11.05%	15.75%	39.34%
Marmousi	high	499	10.20%	13.66%	32.74%
Overthrust	low	499	4.49%	6.40%	28.15%
Overthrust	moderate	491	2.84%	4.25%	21.65%
Overthrust	high	458	2.03%	3.69%	25.23%
Salt	low	470	3.25%	4.49%	10.51%
Salt	moderate	419	**2.70**%	**2.41**%	8.08%
Salt	high	458	10.69%	18.94%	45.03%

The two Salt entries are the pair worth watching. Sample 419 is among the easiest in the whole study — a smooth sediment gradient, and the only animation anywhere in which **PFNO beats FNO** (2.41% against 2.70%; one sample, so noise rather than a real reversal). Sample 458 is the same model at 4× the error, because that column crosses a salt body: the reflection off the boundary is visible in the FD panel, and all three predictions blur it.

Note that the sample ids repeat across models (458 appears three times). They are indices into each arm's own validation set, not a shared identifier.

12.6 Reproducing

```
# Fetch models into operator_data/raw/ (URLs in wave/operator_sim/README.md)
python wave/operator_sim/generate_dataset_real.py --model marmousi \
    --num-samples 512 --seed 42 \
    --out operator_data/wave_operator_marmousi_n512_nx64_nt64_t1_seed42.npz
```

```
# Synthetic arm, now with a converged target
python wave/operator_sim/generate_dataset.py --refine 8 --num-samples 512 \
    --seed 42 \
    --out operator_data/wave_operator_fixedic_r8_n512_nx64_nt64_t1_seed42.npz

# Train (GPU). NOTE --don-latent/--don-hidden: the script defaults are
# 192/384, but every result quoted here uses the larger 256/512 pair.
python wave/operator_sim/train_operators.py --data <npz> --epochs 400 \
    --don-latent 256 --don-hidden 512 --outdir out_<arm>
```

13 Visualization design

Two defects in the original figure were fixed. Both affected interpretability independently of model quality.

1. The reference is now shown. The original compared the three predictions *against each other only*, reporting a pairwise disagreement matrix. This is actively misleading when models are undertrained: a near-zero field looks smooth and plausible, and the disagreement numbers (110–140%) were large without indicating which model, if any, was right. Every panel now sits against the FD reference, and titles carry the per-sample relative L_2.

2. The colour scale is set from the propagating wave. The $t = 0$ pulse has unit amplitude but immediately splits into two waves of ≈ 0.5 each. Measured across all 512 samples:

max $\lvert u\rvert$ over all t		1.00
max $\lvert u\rvert$ for $t > 0$	0.97	*(frame 1 still contains the pulse)*
98th percentile of $\lvert u\rvert$	**0.55**	*(the propagating amplitude)*

Scaling `vmin`/`vmax` to the global maximum therefore spent half the colormap on a single frame and pushed everything afterwards into the pale middle. The scale is now the 98th percentile of $|u|$, computed from the reference and shared across all four field panels. The initial pulse saturates for the first few frames, which is the intended trade.

```
# Colour: propagating wave, not the t=0 pulse (~1.8x taller).
amplitude = max(float(np.percentile(np.abs(reference), 98.0)), 1e-6)
# Line plots keep the full range so the initial pulse is not clipped.
line_amplitude = max(float(np.abs(reference).max()),
                     max(float(np.abs(f).max()) for f in wavefields.values()), 1e-6)
```

3. The series palette was re-stepped. The original blue/teal pair for FNO and DeepONet sat at $\Delta E = 14.0$ under normal colour vision, below the legibility floor — hard to separate even with full colour vision, and worse under simulated deuteranopia. The palette is now ■ FNO `#3573B9`, ■ DeepONet `#B03A5B`, ■ PFNO `#C98A00`, whose worst adjacent pair is $\Delta E = 16.0$ (protanopia) and 22.8 (normal vision). Panel titles are set in ink rather than the series colour; identity is carried by the coloured line in the legend, so no text depends on colour to be readable.

Smaller choices, for the record: the reference curve is drawn thick and pale *underneath* the thin coloured prediction, so overlap stays legible and any deviation reads immediately; the velocity model $c(x)$ is shown so wave behaviour is explicable from the input; and the rendered samples are the median-error sample per family rather than the best, so the animations are representative.

14 The code

Everything lives in `wave/operator_sim/`.

File	Role
`generate_dataset.py`	FD dataset generation (wraps the repo's sampler and solver)
`operator_models.py`	The three architectures, verbatim from the notebook
`train_operators.py`	Training loop, evaluation, prediction export
`render_simulations.py`	Animation rendering
`README.md`	Quick-start summary

The split between training and rendering is deliberate: training writes *no* plots, so the GPU host needs no matplotlib. Training exports predictions to an `.npz`, and rendering happens wherever matplotlib is available.

14.1 Training core

```
opt = torch.optim.AdamW(model.parameters(), lr=a.lr, weight_decay=a.weight_decay)
sched = torch.optim.lr_scheduler.OneCycleLR(
    opt, max_lr=a.lr, total_steps=a.epochs * len(train_loader), pct_start=0.15)

for epoch in range(1, a.epochs + 1):
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        opt.zero_grad(set_to_none=True)
        loss = F.mse_loss(model(xb), yb)
        loss.backward(); opt.step(); sched.step()
    vmse, vrel = evaluate(model)
    if vmse < best_val:                       # keep the best, not the last
        best_val = vmse
        best_state = copy.deepcopy(model.state_dict())
```

Evaluation de-normalizes before measuring, so the reported error is in physical units:

```
@torch.no_grad()
def evaluate(model):
    model.eval(); se = 0.0; n = 0; rels = []
    for xb, yb in val_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        pred, tgt = denorm(model(xb)), denorm(yb)
        se += F.mse_loss(pred, tgt, reduction="sum").item(); n += tgt.numel()
        pf, tf = pred.flatten(1), tgt.flatten(1)
        rel = (torch.linalg.vector_norm(pf - tf, dim=1)
               / torch.linalg.vector_norm(tf, dim=1).clamp_min(1e-12))
        rels.extend((100.0 * rel).cpu().tolist())
    return se / n, float(np.mean(rels))
```

The run exports `operator_wave_predictions_v2.npz` containing x, t, the validation ids and kinds, the E and ρ profiles, the FD target, and one $(102, 64, 64)$ array per model — everything the renderer needs.

14.2 Rendering

`FuncAnimation` over the 64 time slices; only the line data and time cursors update per frame, while the space–time images are drawn once. Frames are then quantized to a 128-colour palette, roughly halving file size with no visible change on these mostly-flat frames. Sample selection defaults to the median-error sample of each family:

```
for kind in ("homogeneous", "two_layer", "layered", "smooth"):
    cand = [i for i in range(len(ids)) if kinds[i] == kind]
    errs = [rel_l2(preds["FNO"][i], target[i]) for i in cand]
    chosen.append(cand[int(np.argsort(errs)[len(errs) // 2])])
```

15 Reproducing end to end

```
# 1. Generate the FD dataset (~2 s)
python wave/operator_sim/generate_dataset.py \
    --num-samples 512 --nx 64 --nt 64 --seed 42 \
    --out operator_data/wave_operator_fixedic_n512_nx64_nt64_t1_seed42.npz

# 2. Train all three operators (~15 min on one H100)
python wave/operator_sim/train_operators.py \
    --data operator_data/wave_operator_fixedic_n512_nx64_nt64_t1_seed42.npz \
    --epochs 400 --batch-size 16 --lr 3e-3 \
    --fno-width 48 --fno-modes 20 --fno-layers 4 \
    --don-latent 256 --don-hidden 512 \
    --pfno-width 24 --pfno-modes 20 --pfno-layers 3 \
    --outdir server_outputs_v2

# 3. Render the GIFs (~5 min per GIF, CPU, matplotlib)
python wave/operator_sim/render_simulations.py \
    --pred server_outputs/operator_wave_predictions_v2.npz \
    --summary server_outputs/operator_wave_summary_v2.json \
    --outdir simulations --fps 12 --dpi 95
```

On a GPU host without matplotlib, run steps 1–2 remotely, copy back `operator_wave_predictions_v2.npz` and `operator_wave_summary_v2.json`, and run step 3 locally.

Step 2 on CPU is impractical — roughly seven hours, dominated by PFNO's 33 sequential 1D branches. For a CPU smoke test use a subset and smaller models (`--epochs 30 --fno-width 16 --fno-modes 12 --fno-layers 2 --don-latent 96 --don-hidden 192 --pfno-width 8 --pfno-modes 12`).

In the notebook. `notebooks/FNO_DeepONet_PFNO_wave_comparison.ipynb` carries the same configuration:

- `QUICK_RUN = True` (default) — 128-sample subset, 30 epochs, small models; about 10 minutes on a laptop CPU. **A smoke test, not converged.**
- `QUICK_RUN = False` — the full 512-sample, 400-epoch configuration above.
- The simulation section sets `USE_PRETRAINED = True` whenever `server_outputs/operator_wave_predictions_v2.npz` exists, so the animation shows the converged GPU results regardless of how the notebook itself was run. Set it to `False` to animate the in-session models.
- `SAVE_COMPARISON_GIF = True` writes a GIF into `simulations/`.

16 Limitations

Stated plainly, so the numbers are not over-read.

- **One seed, one split.** No error bars, no seed averaging. Differences of a few tenths of a percent between configurations are not meaningful; the order-of-magnitude gaps between architectures are.
- **Fixed initial condition.** Every sample uses the same pulse ($\sigma_g = 0.1$, $x_0 = 0$). The operators learn a velocity-model → wavefield map, *not* a general initial-condition → wavefield map. `run_fno_baseline.py --random-ic` exists if that is wanted, but nothing here has been trained or evaluated that way.
- **Fixed 64×64 grid.** FNO and PFNO are in principle discretization-invariant, but zero-shot super-resolution has not been tested here.

- **In-distribution evaluation only.** Validation materials come from the same four families as training. Nothing tests transfer to the project's canonical `Homogeneous` / `TwoLayer` / `MultiLayer` classes in `wave/materials.py`, or to sharper contrasts than the sampler produces.
- **Not a paper reproduction.** The three architectures are small, readable reimplementations sized to train in minutes, not faithful reproductions of the Li et al. or Lu et al. configurations. The DeepONet result should be read as "this rank-p separable form, at this size, on this data" — not as a general statement about DeepONet.
- **Parameter counts are not matched.** FNO has 7.4 M parameters against DeepONet's 0.89 M and PFNO's 1.2 M. Some of FNO's advantage is capacity, not purely architecture. A parameter-matched comparison has not been run.
- **DeepONet is probably under-served in the forced arm.** A source whose *position* varies is close to the worst case for a fixed separable basis: representing a translating feature needs many basis functions. Its capacity was not retuned for that arm, so 91.8% should be read as "this rank-p form, at this size, on this problem" rather than as a ceiling.
- **The IC/BC residuals have limited resolving power.** They are evaluated by finite differences on the coarse 64×80 output grid, and the finite-difference reference's own relative boundary residual is 0.126 — not small. Differences well below that floor should not be read as meaningful, and as Section 10 shows, a low residual can indicate a collapsed field rather than good physics.
- **The two arms are not directly comparable.** Different grid (64×64 vs 64×80), time window ($t \leq 1$ vs $t \leq 2$), and driving mechanism. Cross-arm error comparisons are indicative only.
- **The physics-loss sweep never found its breaking point.** Every metric was still improving at $\lambda = 10$, the largest weight tried, so the optimum is not bracketed and $\lambda = 1$ is a safe choice rather than a tuned one. The sweep is also FNO-only and single-seed; the three-model comparison reuses one weight across architectures whose residual magnitudes differ by roughly $6\times$.

17 File inventory

Code — `wave/operator_sim/`
Unforced arm: `generate_dataset.py`, `operator_models.py`, `train_operators.py`, `render_simulations.py`, `README.md`. Forced arm: `fd_source.py` (forced FD solver), `generate_dataset_source.py`, `train_source.py`, `physics_metrics.py` (IC/BC residuals, numpy), `physics_losses_torch.py` (the same terms, differentiable).

Data
`operator_data/wave_operator_source_n512_nx64_nt80_t2_seed42.npz` (forced arm, 6 input channels), and
`operator_data/wave_operator_fixedic_n512_nx64_nt64_t1_seed42.npz`
($512 \times 5 \times 64 \times 64$ inputs, $512 \times 1 \times 64 \times 64$ outputs, 8.5 MB)

Forced-arm artefacts — `server_outputs_source/`
`operator_source_predictions.npz` (validation ids/kinds, E, ρ, $s(x)$, $w(t)$, x_s, f, FD target, per-model predictions); `operator_source_summary.json` (metrics *and* the IC/BC residual table); `histories_source.json`; `train_src.log`.

Physics-informed artefacts — `server_outputs_pinn/`
Predictions, summary and histories for the $\lambda = 1$ run, plus `lambda_sweep.json` holding the full weight sweep.

Trained artefacts — `server_outputs/`
`operator_wave_predictions_v2.npz` (validation ids/kinds, E, ρ, FD target, per-model

predictions); `operator_wave_summary_v2.json` (device, dataset shape, full arg list, per-model metrics); `histories_v2.json` (per-epoch curves); `train_v2.log`. The superseded undertrained run is kept as `operator_wave_predictions.npz` and `operator_wave_summary.json`.

Animations — `simulations/`
Four GIFs for the unforced arm; four more for the forced arm under `simulations/source_term/`; plus `superseded_undertrained/` holding the three originals.

Documentation — `docs/`
`operator_simulations.md` (this document in Markdown), `operator_simulations.tex`, `operator_simulations.pdf`, `figures/`

Model checkpoints (`FNO_state.pt` at 59 MB, `DeepONet_state.pt`, `PFNO_state.pt`) remain on the training host under `~/sciml_wave_sim/v2/server_outputs_v2/` and were not copied back.

Reference papers

Included under `fno paper/`:

- `01_Fourier_Neural_Operator_for_Parametric_PDEs.pdf` — Li et al., *Fourier Neural Operator for Parametric Partial Differential Equations.*
- `DeepONet.pdf` — Lu, Jin, Karniadakis, *Learning nonlinear operators...*
- `2209.12340v3.pdf` — Li et al., *Solving Seismic Wave Equations on Variable Velocity Models with Fourier Neural Operator* (the paralleled-FNO reference).